CS4475 Fall 2026 Lab Report

Gael Sustaita
gsustaita4@miners.utep.edu

Submitted on 11/08/2026

PA4: Parallel Matrix Operations using MPI

Repository: https://github.com/gael68/pa4-mpi-matvec (branch main). The first commit contains the provided starter files unmodified, so the history shows every change I made. This report is committed as PA4 Report.pdf at the top of the repository.

Environment. All timings come from Bridges-2 regular-memory nodes (2 × AMD EPYC 7742, 64 cores per socket, 256 GB) using the RM-shared partition with 16 cores, Open MPI 5.0.8 and gcc 13.3.1 at `-O2` (the provided Makefile). Every run was repeated (5 times unless noted) and I report averages. The raw Slurm output for every number in this report is in `results/`. `analyze.py` regenerates every table and graph from those files, and `pa4-data.xlsx` has all the data, one sheet per experiment.

File	Purpose
`mvm.c`	Task 1: the skeleton with the missing `MPI_Allgather` filled in
`mvm-driver.c`, `Makefile`	Provided, unmodified
`mpi_submit`	Task 4 batch script (modified from the provided one)
`task1.sbatch`, `task2.sbatch`, `common.sh`	Batch scripts for the Task 1 correctness runs and Task 2 sequential timings
`transpose.c`, `transpose.sbatch`	Extra credit: distributed transpose and its batch script
`commbench.c`	Measures t_s and t_w (ping-pong) and the allgather cost for the Task 3 model
`mvm_timed.c`, `deep.sbatch`	Copy of mvm.c that times the allgather and the local product separately
`extras.sbatch`, `placement.sbatch`	Hardware info, rank-placement experiments, scaling past p = 8
`analyze.py`, `figures/`, `pa4-data.xlsx`	Builds the tables, graphs and spreadsheet from `results/`

Task 1. (10 pts) Implementing the MPI communication in mvm.c

The driver distributes A by rows: each of the p processes receives `nlocal = n/p` complete rows of A and the matching `nlocal` entries of x. Row i of y is the dot product of row i of A with *all* of x, but each process holds only its slice of x. So before the local loop every process needs every other process's slice. The missing operation is therefore an all-gather: each process contributes `nlocal` doubles and receives the whole vector, in rank order, which is also the order of the rows.

```
    /* Gather entire vector x on each processor */
    /********************************************/
    MPI_Allgather(x, nlocal, MPI_DOUBLE, xbuf, nlocal, MPI_DOUBLE, comm);
    /********************************************/
```

That is the only change to the skeleton (`mvm.c:26`). Gather followed by Bcast would also work, but it needs two collectives and funnels the whole vector through rank 0. Allgather does it in one collective that the library can implement in log p steps.

Testing. `task1.sbatch` runs n = 8, 32, 1024 and 32000 on 1, 2, 4 and 8 processes on a Bridges-2 compute node. The driver recomputes y sequentially on rank 0 and prints a `discrepancy` line for every element that differs, and the script counts those lines. As an independent check, the driver's inputs (`a[i][j] = 2i + j`, `x[j] = j`) give a closed form for the last element, $y[n-1] = 2(n-1)\cdot n(n-1)/2 + (n-1)n(2n-1)/6$, and I compared it with what each run printed.

n	p	discrepancies	y[n−1] printed	matches closed form	time (s)
8	1	0	532	yes	0.000005
8	2	0	532	yes	0.000019
8	4	0	532	yes	0.000032
8	8	0	532	yes	0.000083
32	1	0	41168	yes	0.000004
32	2	0	41168	yes	0.000037
32	4	0	41168	yes	0.000015
32	8	0	41168	yes	0.000034
1024	1	0	1429035520	yes	0.001029
1024	2	0	1429035520	yes	0.000555
1024	4	0	1429035520	yes	0.000283
1024	8	0	1429035520	yes	0.000204

Table 1. Small inputs (results/task1-50358715.out).

n	p	discrepancies	y[n−1] printed	matches closed form	time (s)
32000	1	0	43688106704000	yes	1.029128
32000	2	0	43688106704000	yes	0.511679
32000	4	0	43688106704000	yes	0.272106
32000	8	0	43688106704000	yes	0.243680

Table 2. The n = 32000 input used for the parallel timings (same file).

Every run is correct. The floating-point comparison can be exact because all inputs are small integers, each row is summed in the same order by the parallel code and the check, and the largest value (4.4×10^{13}) is below 2^{53}. For n = 8 and 32, more processes are *slower*: there are only 64 or 1024 multiply-adds, and the microseconds of collective overhead dominate. From n = 1024 the work is large enough for parallelism to pay off.

Task 2. (10 pts) Sequential execution analysis

Theoretical analysis. For each of the n rows the inner loop does n multiply-adds, so a sequential matrix-vector product does n^2 multiply-adds ($2n^2$ floating-point operations) and reads each of the n^2 matrix entries exactly once, plus O(n) work for x and y. The time is therefore

$$T_1(n) = t_c \cdot n^2 = \Theta(n^2)$$

where t_c is the cost of one multiply-add, including fetching its matrix entry. The model predicts that doubling n multiplies the time by 4.

Measurements. `task2.sbatch` runs mvm with one task, 5 times per size. The time is the one the driver prints, which covers only `MatrixVectorMultiply`.

n	avg (s)	min (s)	stdev (s)	avg / n^2 (ns)	ratio to previous n
1000	0.000985	0.000963	0.000023	0.985	–
2000	0.003992	0.003949	0.000036	0.998	4.05
4000	0.016655	0.015803	0.001038	1.041	4.17
8000	0.065428	0.063374	0.001553	1.022	3.93
16000	0.260382	0.251852	0.006175	1.017	3.98
32000	1.030061	1.028398	0.001805	1.006	3.96

Table 3. Sequential timings, one task (results/task2-50358716.out).

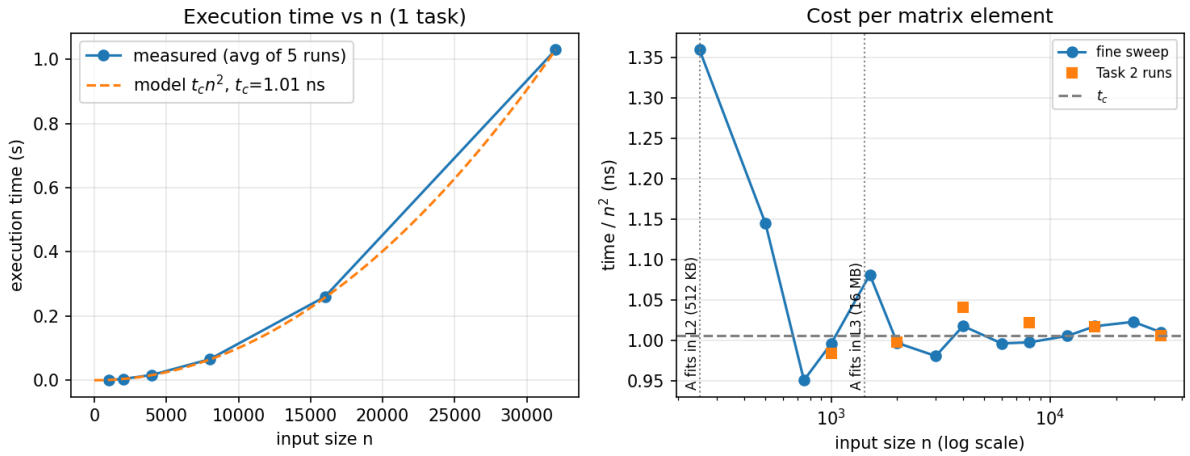


Figure 1. Left: execution time vs input size with the model $t_c \cdot n^2$ (t_c fitted at n = 32000). Right: time ÷ n^2, which makes deviations from the model visible; includes a finer sweep from deep-50358765.out (3 runs per size).

Comparison with the model. The results follow the model closely. Each doubling of n multiplies the time by 3.93–4.17, against a predicted 4, and the time per matrix element is flat at 1.0 ± 0.04 ns from n = 1000 to 32000. Fitting at n = 32000 gives **t_c = 1.006 ns**. The deviations I found:

- **The constant reflects memory bandwidth, not arithmetic.** 1 ns per element means 8 bytes of A every nanosecond, about 8 GB/s, which is roughly what one EPYC core can stream from DRAM. A core at 2.25–3.4 GHz could do several multiply-adds per nanosecond, so the loop waits on memory. The asymptotic model is still right, but t_c is set by the memory system. This matters again in Task 5.
- **Small n costs more per element** (1.36 ns at n = 250, 1.15 ns at n = 500). `MatrixVectorMultiply` has fixed costs that don't grow with n^2: allocating `xbuf`, the `MPI_Allgather` call (about 3 µs even with one process) and the timer calls. At n = 250 the whole product takes only 0.08 ms, so those costs are visible. A complete model would be $t_c \cdot n^2 + c_0$.

- **A small bump when A outgrows the L3 cache.** At n = 750–1000, A (4.5–8 MB) fits in the 16 MB L3 and is partly still cached from the scatter (0.95–1.0 ns per element). At n = 1500 (18 MB) it no longer fits and the per-element cost rises to 1.08 ns before settling at the DRAM-streaming rate.
- **Run-to-run variation** is small: the largest standard deviation is 6 ms at n = 16000 (2.4%).

Task 3. (10 pts) Analytical model of the parallel runtime

The model covers only `MatrixVectorMultiply`: one `MPI_Allgather` followed by the local product. As the handout asks, the driver's scatter, gather and correctness check are excluded. That is also exactly the span the driver's timer measures.

Computation

Each process owns n/p rows of length n and does one multiply-add per entry. All processes run at the same time, so

$$T_{comp} = t_c \cdot n \cdot (n/p) = t_c \cdot n^2 / p$$

Communication

Each process contributes m = n/p words and needs the other (p − 1)·n/p. With recursive doubling (the hypercube all-gather) there are $\log_2 p$ steps. In step i each process exchanges the 2^{i-1}·n/p words it has collected so far with one partner, so each step costs $t_s + t_w \cdot 2^{i-1} \cdot n/p$:

$$T_{comm} = \sum_{i=1}^{\log p} \left(t_s + t_w \cdot 2^{i-1} \cdot n/p \right) = t_s \cdot \log_2 p + t_w \cdot n \cdot (p-1)/p$$

The t_s term counts message startups, one per step. The t_w term counts the words each process receives: almost the whole vector, n(p − 1)/p ≈ n, whatever p is. A ring all-gather moves the same number of words but pays (p − 1)·t_s in startups.

Total, speedup and efficiency

$$T_p = t_c \cdot n^2/p + t_s \cdot \log_2 p + t_w \cdot n \cdot (p-1)/p$$

$$S = T_1 / T_p = t_c\, n^2 / T_p \qquad E = S / p$$

- **Overhead:** $T_o = p \cdot T_p - T_1 = t_s \cdot p \log_2 p + t_w \cdot n(p-1)$.
- **Cost-optimal** ($p \cdot T_p = \Theta(n^2)$) as long as $p = O(n)$.
- **Isoefficiency:** keeping E constant requires $t_c\, n^2 \propto t_w\, n\, p$, so n must grow in proportion to p. The work $W = n^2$ must grow as $\Theta(p^2)$.
- Computation is $\Theta(n^2/p)$ but communication only $\Theta(\log p + n)$, so for large n the model predicts near-linear speedup.

Parameters measured on Bridges-2

To make the model quantitative I measured the parameters on the same hardware. `commbench.c` bounces messages of 8 bytes to 32 MB between two ranks (ping-pong). The one-way time is $t_s + t_w \cdot m$, so t_s is the time for the smallest message and t_w is the slope for large messages.

Parameter	Value	How it was measured
t_c	1.006 ns	Task 2: T(32000) / 32000^2

Parameter	Value	How it was measured
t_s (ranks on the same socket)	0.144 µs	ping-pong, 8-byte message (extras-50358718.out)
t_w (ranks on the same socket)	0.550 ns per 8-byte word	ping-pong slope, messages ≥ 512 KB
t_s, t_w (different sockets)	0.649 µs, 0.853 ns	same test with ranks on different sockets

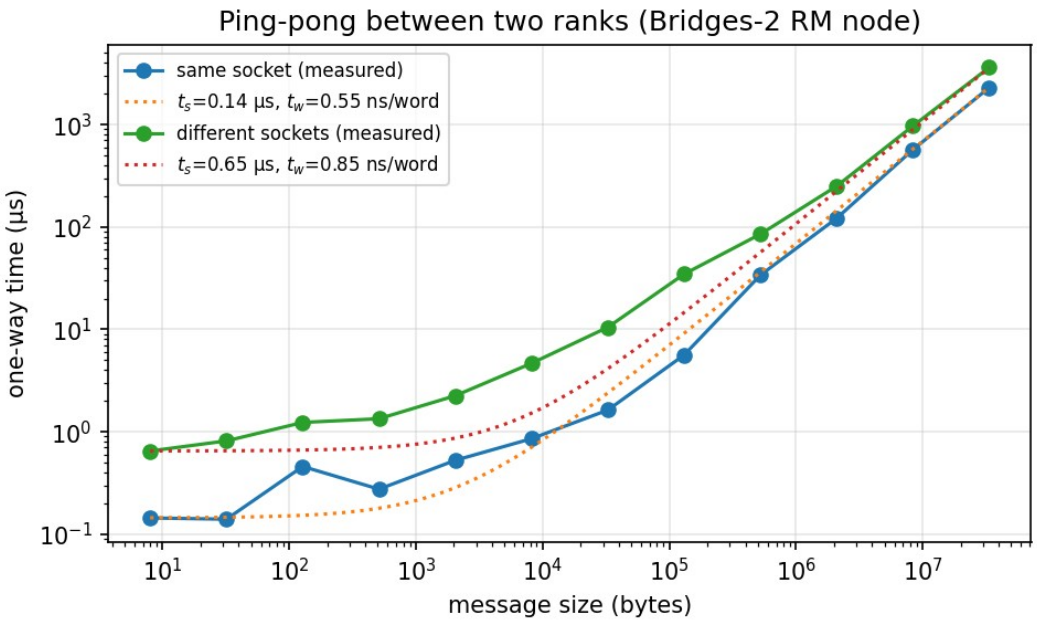


Figure 2. One-way message time between two ranks on a Bridges-2 RM node, with the fitted t_s + t_w·m lines.

Prediction for n = 32000, p = 8: T_comp = 1.006 ns × $32000^2/8$ = 0.129 s, and T_comm = 0.144 µs × 3 + 0.550 ns × 28000 ≈ 16 µs. Communication should be about 0.01% of the runtime, so the model predicts a speedup of 8.0. At n = 1000 and p = 8 the computation is only 0.13 ms, so the same communication becomes a noticeable fraction. That is the isoefficiency argument in numbers.

Task 4. (10 pts) Batch job script and execution

I started from the provided `mpi_submit` and changed only what was needed to run correctly and produce clean data:

Change	Why
`-p RM-shared`, `--tasks-per-node=16`	At p = 1, rank 0 holds both the full A (8.2 GB) and its scattered copy (8.2 GB). RM-shared allocates about 2 GB per core, so 8 cores (16 GB) is too little memory; 16 cores gives 32 GB. RM-shared is also much cheaper than a whole 128-core RM node.
`cd "$SLURM_SUBMIT_DIR"`	The provided script changed to the TA's own directory.
`export HOSTNAME=...`	Bridges-2's gcc module fails to load if HOSTNAME is unset.
`set -x` after `module load`	With it before, the module system's trace adds hundreds of lines to the output.
5 runs per process count, `-J`, `-o task4-%j.out`	Averages out noise; names the job and the output file.

The job script, `mpi_submit`:

```
#!/bin/bash
#SBATCH -J pa4-task4
#SBATCH -p RM-shared
#SBATCH -N 1
# 16 cores for the ~2 GB/core memory: rank 0 holds two 8 GB copies of A at n=32000
#SBATCH --tasks-per-node=16
#SBATCH -t 0:30:00
#SBATCH -o task4-%j.out

cd "$SLURM_SUBMIT_DIR"
export HOSTNAME=${HOSTNAME:-$(hostname -s)}
module load openmpi

# echo commands to stdout (after module load, whose trace is hundreds of lines)
set -x
make mvm

RUNS=5
for np in 1 2 4 8; do
    for r in $(seq 1 $RUNS); do
        echo "Running with $np processes (run $r)..."
        mpirun -n $np ./mvm 32000
    done
done

echo "Job completed"
```

I submitted it with `sbatch mpi_submit` (job 50358720, node r195, 1 min 47 s). The complete output file is in the appendix and in `results/task4-50358720.out`.

Correctness of the job's results. All 20 runs printed y[31999] = 43688106704000, which equals the closed form from Task 1, and the output contains no `discrepancy` lines (`grep -c discrepancy results/task4-50358720.out` prints 0). The timings are summarized in Table 4 in Task 5.

Task 5. (10 pts) Speedup analysis

p	avg (s)	min (s)	stdev (s)	speedup T_1/T_p	efficiency	Task 3 model speedup
1	1.042269	1.037225	0.0042	1.00	1.00	1.00
2	0.516359	0.513244	0.0023	2.02	1.01	2.00
4	0.278159	0.272366	0.0076	3.75	0.94	4.00
8	0.240932	0.232616	0.0058	4.33	0.54	8.00

Table 4. n = 32000, 5 runs per process count (results/task4-50358720.out). Speedups use the average times.

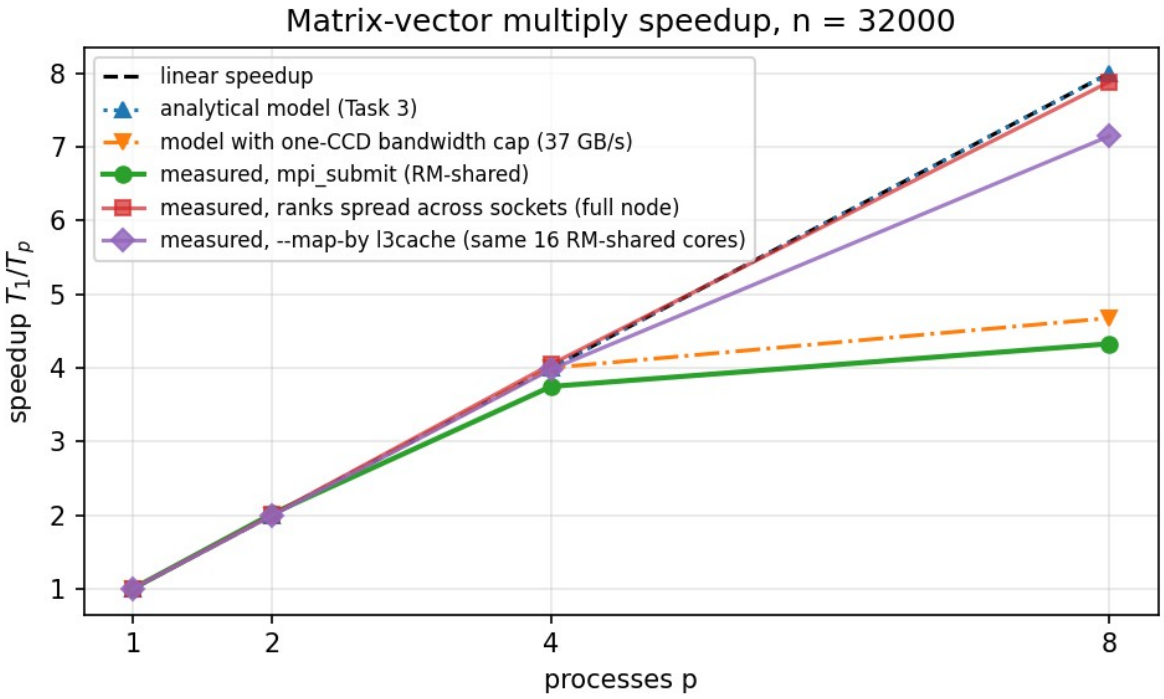


Figure 3. Measured speedup (green) against linear speedup, the Task 3 model, and the refined model and placement experiments explained below.

Comparison with the model. Up to p = 4 the measurements follow the model: 2.02 at p = 2 and 3.75 at p = 4, against 2 and 4. At p = 8 the measured speedup flattens to 4.33 (efficiency 0.54), while the model predicts 8. I ran three experiments to find the cause.

1. Communication is not the cause. `mvm_timed.c` is a copy of `mvm.c` that times the allgather and the local product separately on every rank. At n = 32000 the allgather is under 0.5% of the time at every p (Figure 4). The computation itself is what stops scaling: the slowest rank's local product takes 0.283 s at p = 4 and still 0.237 s at p = 8, against 0.129 s predicted.

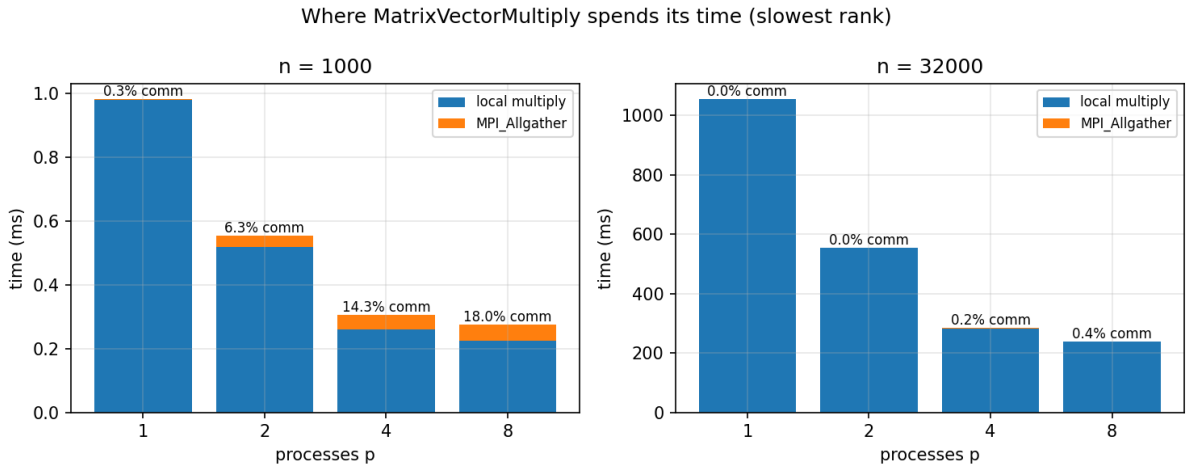


Figure 4. Time inside MatrixVectorMultiply for the slowest rank (deep-50358765.out, 3 runs each). Communication is visible at n = 1000 and negligible at n = 32000.

The allgather does take longer than the model predicts: 0.2–0.8 ms against about 16 µs. Most of that is waiting rather than data transfer. The allgather is the first synchronizing call after the driver's scatter, so ranks that arrive early wait for rank 0, which arrives last. Measured back-to-back on its own, the allgather takes 127 µs at p = 8. Either way it is too small to explain a 0.1 s gap.

2. Memory bandwidth is the cause. Task 2 showed the loop is limited by memory bandwidth: each process streams its rows of A from DRAM at about 8 GB/s. `mpirun --report-bindings` shows that in the 16-core RM-shared allocation the 8 ranks were placed on cores 32–39 (`deep-50358765.out`). On an EPYC 7742 those 8 cores form one chiplet (CCD), and the CCD reaches memory through a single link. 8 ranks × 8 GB/s = 64 GB/s, but on an exclusive node I measured that packing 8 ranks onto adjacent cores saturates at **37.2 GB/s**, while spreading them across sockets reaches 64 GB/s and a speedup of 7.9 (Figure 5).

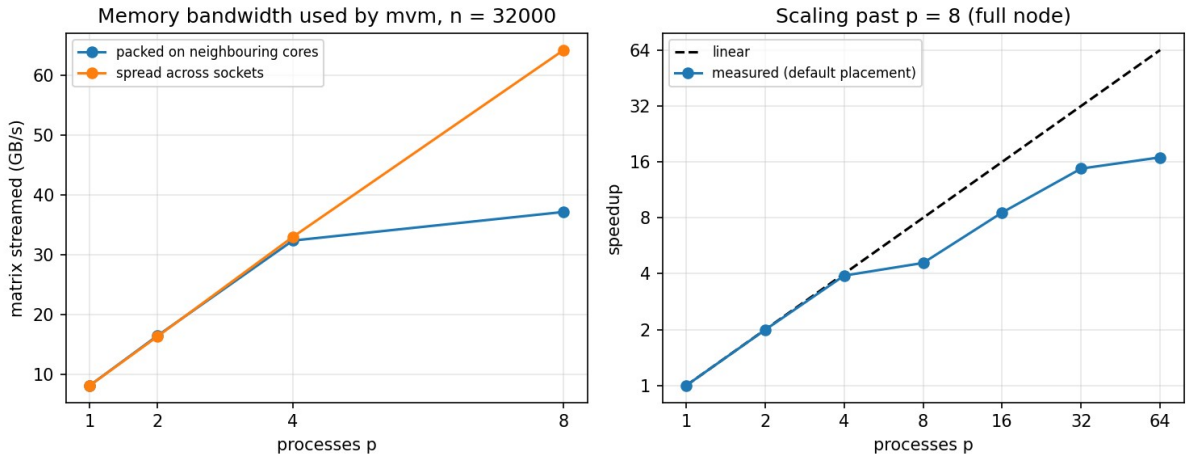


Figure 5. Left: bytes of A streamed per second with ranks packed on neighbouring cores vs spread across sockets (exclusive RM node, extras-50358718.out). Right: on a full node, speedup keeps growing past p = 8 as new ranks reach fresh chiplets.

3. Proof in the same allocation. To rule out everything except placement, I reran the program in a new 16-core RM-shared job and changed only the mapping flag (`placement-50358821.out`). With `--map-by l3cache`, mpirun places the 8 ranks round-robin over the 4-core L3 clusters (cores 32, 33, 36, 37, 40, 41, 44, 51), so no memory path is shared by more than two ranks.

p	--map-by core (default)	speedup	--map-by l3cache	speedup
1	1.0272 s	1.00	1.0120 s	1.00
2	0.5235 s	1.96	0.5073 s	1.99
4	0.2774 s	3.70	0.2543 s	3.98
8	0.2227 s	4.61	0.1416 s	7.15

Table 5. Same program, same kind of allocation, different rank placement (3 runs each).

Refined model. The Task 3 model treats t_c as a constant, but t_c is really the cost of moving 8 bytes from memory. When the ranks share one memory link of bandwidth B, the computation cannot go faster than the link delivers:

$$T_p = n^2 \cdot \max(\, t_c / p\,,\ 8\ \text{bytes} / B\,) + t_s \cdot \log_2 p + t_w \cdot n \cdot (p-1)/p$$

With B = 37.2 GB/s for one CCD, this predicts 4.68 at p = 8, against 4.33 measured (orange line in Figure 3). With the ranks spread, B is large enough that the original model applies, and the measured 7.15–7.9 approaches its prediction of 8.

Discussion. The Task 3 model is right that communication is negligible for this problem size: the algorithm moves O(n) words but does $O(n^2/p)$ work. It is too optimistic because it assumes each process has its own memory bandwidth. On a shared-memory node the processes compete for the same memory links, and where mpirun places them decides how much bandwidth each gets. The small-n runs show the other side of the model. At n = 1000 and p = 8, communication is 18% of the time (Figure 4), exactly the regime where the isoefficiency analysis says n is too small for that many processes. The 2.02 speedup at p = 2 is within measurement noise; each half of A is still 4 GB, far larger than any cache.

Extra Credit. (25 pts) Parallel matrix transpose

Implementation (transpose.c)

The n × n matrix of doubles uses the same 1-D row-block decomposition as mvm. Each of the p processes owns b = n/p consecutive rows and initializes its own part from global indices, `A[i][j] = i·n + j`, so no process ever holds the whole matrix. A process's rows split into p blocks of b × b. Block k of process r must become, transposed, block r of process k. The transpose has three phases, each timed separately:

1. **Local transpose (pack)** (`transpose.c:75–87`). Each b × b block is transposed into a send buffer, with block k at offset $k \cdot b^2$. Rows of A are read sequentially, but the writes go down columns, so the loop is tiled (64 × 64 by default) to keep the written cache lines resident.
2. **Exchange** (`transpose.c:90`). `MPI_Alltoall` sends block k to process k and receives one block from every process. This is all-to-all personalized communication: every process has a *different* block for every other process. MPI_Alltoall is the collective designed for exactly this pattern, and it lets the library choose the algorithm (pairwise exchange for large messages). Point-to-point Send/Recv would need careful ordering to avoid deadlock, and Alltoallv is only needed when block sizes differ.
3. **Unpack** (`transpose.c:93–98`). Received block k becomes columns k·b … (k+1)·b − 1 of the local rows of A^T, one `memcpy` per row.

Afterwards every process checks every element (`B[i][j] == A[j][i]`, `transpose.c:112–117`), and rank 0 reports `correct` or `FAILED`. Phase times are reduced with MPI_MAX, because a phase is only as fast as its slowest rank. Correctness on Bridges-2 (results/transpose-50358804.out): n = 8 is correct on 1, 2, 4 and 8 processes, with the printed rows of A^T (0 8 16 … 56 / 1 9 17 … 57 / …); every timed run reports `correct`; n = 10 on 4 processes is rejected with a usage message. Locally I also tested p = 3 and tile sizes that don't divide b.

Analytical model

Let t_m be the cost of moving one element in the strided local transpose and t_u the cost per element of the contiguous unpack copy. Each process holds n^2/p elements:

- **Local data movement:** pack ($t_m \cdot n^2/p$) plus unpack ($t_u \cdot n^2/p$). Every local element is moved twice.
- **Network communication:** with pairwise exchange, each process sends one b × b block (n^2/p^2 words) to each of the other p − 1 processes: $(p-1)\cdot(t_s + t_w \cdot n^2/p^2)$.

$$T_p = (t_m + t_u) \cdot n^2/p + (p-1) \cdot t_s + t_w \cdot n^2 (p-1) / p^2$$

Unlike mvm, the communication term has the same order as the local work (both $\Theta(n^2/p)$), because every element that is moved locally also crosses the network once (except the diagonal blocks). There is no compute-heavy term to hide communication behind, so the speedup is at most linear and falls below it whenever t_w is comparable to t_m + t_u. For p = 1 the alltoall is a local copy of the whole matrix.

Timing results

n = 16000 (each matrix is 2 GB; at p = 1 the four arrays A, send buffer, receive buffer and A^T take 8 GB). RM-shared, 16 cores, 5 repetitions each, average of the slowest rank per phase:

p	pack (s)	alltoall (s)	unpack (s)	total (s)	speedup	untiled total (s)	untiled speedup
1	0.9136	0.1679	0.2486	1.3301	1.00	1.5264	1.00
2	0.2963	0.1323	0.1727	0.5999	2.22	0.8224	1.86
4	0.2340	0.1284	0.1738	0.5336	2.49	0.8184	1.87
8	0.1701	0.1386	0.1672	0.4730	2.81	0.5066	3.01

Table 6. Distributed transpose, n = 16000 (results/transpose-50358804.out). The tiled version (tile 64) is the default.

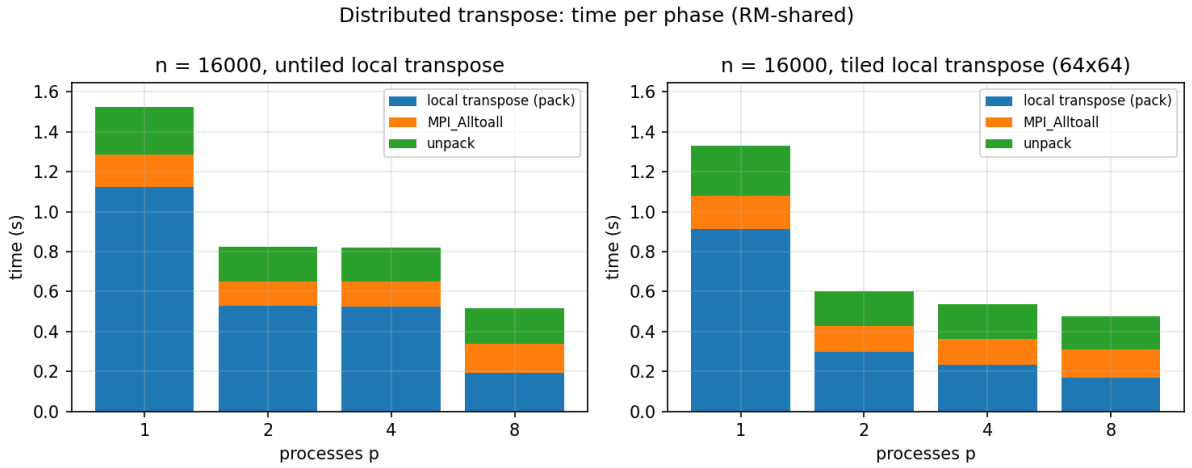


Figure 6. Time per phase, untiled vs tiled local transpose. The pack shrinks with p; alltoall and unpack stay nearly constant.

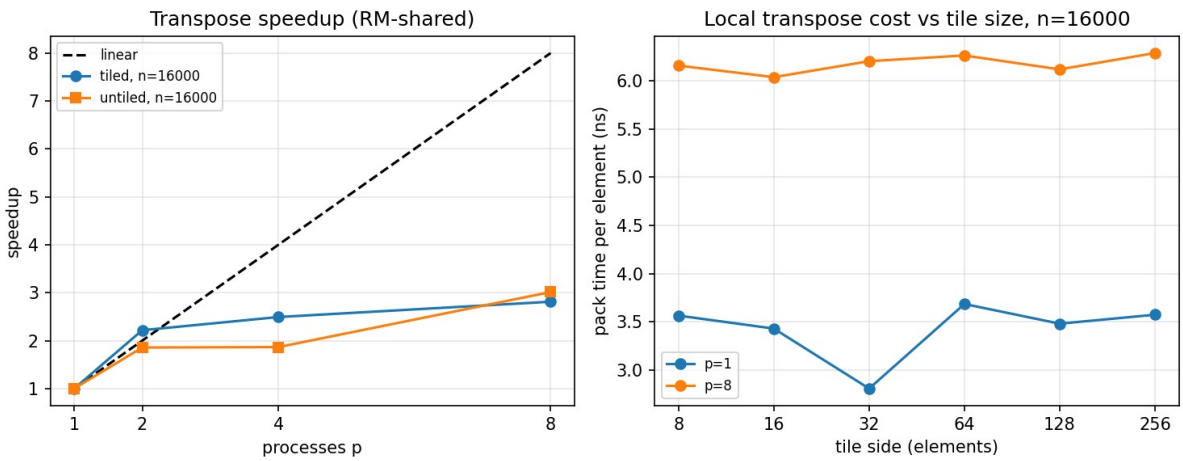


Figure 7. Left: transpose speedup. Right: pack cost per element vs tile size, at p = 1 and p = 8.

Discussion against the model

Local data movement. The pack phase behaves as modelled: it shrinks from 0.91 s to 0.17 s as the n^2/p elements per process shrink. Its per-element cost t_m is 3.6 ns at p = 1, several times the 1 ns per element of the sequential stream in mvm, because the column-order writes touch a new cache line (and often a new page) on every element. Tiling cut the p = 1 pack time by 19% (1.125 s → 0.914 s), and the tile sweep shows the best tile is 32 × 32 (2.8 ns per element), which fits comfortably in the 32 KB L1. At p = 8 tiling makes no difference: there, t_m is limited by the memory bandwidth the 8 ranks share, not by locality. The unpack is a contiguous copy and costs t_u ≈ 1 ns per element at p = 1, as expected for a stream.

Network communication. The alltoall does not shrink with p: 0.13–0.17 s at every p. With the ping-pong t_w, the model predicts 7 × (0.14 µs + 0.55 ns × 4,000,000) ≈ 15 ms at p = 8, but I measured 139 ms. The ping-pong measured one pair of ranks with the memory system to themselves. In an alltoall, all 8 ranks copy through shared memory at the same time, and all 8 sit on the one CCD whose memory link I measured at about 37 GB/s in Task 5. The total traffic is $8n^2(p-1)/p \approx 1.8$ GB, and shared-memory MPI copies it twice (into and out of a shared buffer): about 3.6 GB ÷ 37 GB/s ≈ 97 ms, close to the measurement. On a single node, then, the effective t_w is not a constant: it grows with p because the p processes share one memory link, and that is why the speedup tops out around 2.8. The unpack stays near 0.17 s for the same reason.

Evidence for that explanation. (1) Spreading the 8 ranks over L3 clusters in the same allocation (`--map-by l3cache`, `placement-50358821.out`) cut the p = 8 transpose from 0.498 s to 0.300 s, with alltoall 0.147 → 0.106 s and unpack 0.177 → 0.102 s. (2) On an exclusive node with n = 32000 (8 GB per matrix, extras-50358718.out), where ranks beyond 8 reach new chiplets, the untiled transpose kept speeding up: 2.56 at p = 2, 4.05 at p = 8 and 8.10 at p = 16. Across multiple nodes, t_w would be the network's rate, and the model's prediction that communication is the same order as the local work would hold, with the network as the larger term.

Resources and AI-use disclosure

Resources: the PA4 handout and starter code (mvm-driver.c, mvm-skeleton.c, Makefile, mpi_submit); the Bridges-2 User Guide (PSC); the Open MPI 5.0 documentation for MPI_Allgather, MPI_Alltoall and

the --map-by/--report-bindings options; course material on collective-communication cost models (recursive-doubling all-gather, all-to-all personalized exchange) and isoefficiency.

[Gael: please review and edit this paragraph so it accurately describes how you used AI, then change it to black text.]

I used Claude (Anthropic's Claude Code, model Claude Opus 5.5) as an AI assistant for this assignment. Through an SSH connection I opened to Bridges-2, Claude copied the starter files, wrote the missing MPI_Allgather in mvm.c, wrote the Slurm batch scripts (task1.sbatch, task2.sbatch, the modified mpi_submit, transpose.sbatch, extras.sbatch, deep.sbatch, placement.sbatch), submitted the jobs on my allocation, and copied the output files back. Claude wrote the extra-credit program transpose.c (including the tiled local transpose), the measurement programs commbench.c and mvm_timed.c, and analyze.py, which produced the graphs and the spreadsheet. Claude derived the analytical models in Tasks 2, 3 and the extra credit, investigated the speedup plateau (rank bindings, bandwidth and placement experiments), and drafted this report and a demo walkthrough. It also created the GitHub repository and made the commits. All numbers in this report come from the Bridges-2 output files in results/.

Appendix. Task 4 output file (results/task4-50358720.out)

```
+ make mvm
make: 'mvm' is up to date.
+ RUNS=5
+ for np in 1 2 4 8
++ seq 1 5
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 1 processes (run 1)...'
Running with 1 processes (run 1)...
+ mpirun -n 1 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 1.037225 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 1 processes (run 2)...'
Running with 1 processes (run 2)...
+ mpirun -n 1 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 1.046180 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 1 processes (run 3)...'
Running with 1 processes (run 3)...
+ mpirun -n 1 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 1.046355 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 1 processes (run 4)...'
Running with 1 processes (run 4)...
+ mpirun -n 1 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 1.038888 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 1 processes (run 5)...'
Running with 1 processes (run 5)...
+ mpirun -n 1 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 1.042695 sec
+ for np in 1 2 4 8
++ seq 1 5
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 2 processes (run 1)...'
Running with 2 processes (run 1)...
+ mpirun -n 2 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.516549 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 2 processes (run 2)...'
Running with 2 processes (run 2)...
+ mpirun -n 2 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.515117 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 2 processes (run 3)...'
Running with 2 processes (run 3)...
+ mpirun -n 2 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.513244 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 2 processes (run 4)...'
Running with 2 processes (run 4)...
+ mpirun -n 2 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.517611 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 2 processes (run 5)...'
Running with 2 processes (run 5)...
+ mpirun -n 2 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.519272 sec
+ for np in 1 2 4 8
++ seq 1 5
+ for r in $(seq 1 $RUNS)
```

```
+ echo 'Running with 4 processes (run 1)...'
Running with 4 processes (run 1)...
+ mpirun -n 4 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.272366 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 4 processes (run 2)...'
Running with 4 processes (run 2)...
+ mpirun -n 4 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.273362 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 4 processes (run 3)...'
Running with 4 processes (run 3)...
+ mpirun -n 4 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.283679 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 4 processes (run 4)...'
Running with 4 processes (run 4)...
+ mpirun -n 4 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.288808 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 4 processes (run 5)...'
Running with 4 processes (run 5)...
+ mpirun -n 4 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.272582 sec
+ for np in 1 2 4 8
++ seq 1 5
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 8 processes (run 1)...'
Running with 8 processes (run 1)...
+ mpirun -n 8 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.243951 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 8 processes (run 2)...'
Running with 8 processes (run 2)...
+ mpirun -n 8 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.232616 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 8 processes (run 3)...'
Running with 8 processes (run 3)...
+ mpirun -n 8 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.237474 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 8 processes (run 4)...'
Running with 8 processes (run 4)...
+ mpirun -n 8 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.246677 sec
+ for r in $(seq 1 $RUNS)
+ echo 'Running with 8 processes (run 5)...'
Running with 8 processes (run 5)...
+ mpirun -n 8 ./mvm 32000
Done with mvm, y[31999] = 43688106704000.000000
Parallel matrix vector multiplication time: 0.243942 sec
+ echo 'Job completed'
Job completed
```